Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5 -

Obtained Marks: _____

Artificial Neural Networks

Lab Task # 03

Last date of Submission: October 31st ,2026

Submitted To: Muhammad Hasaan Mujtaba

Student Name: Muhammad Hassaan

Reg. Number: 24108249

Question solution:

jupyter Untitled1 Last Checkpoint: 23 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
[1]: import numpy as np
     import pandas as pd
     import matplotlib.pyplot as plt

[3]: df = pd.read_csv(r"C:\Users\92335\Downloads\data (1).csv")

[4]: df = df.drop(columns=["id", "Unnamed: 32"], errors="ignore")

[5]: df = df.dropna()
```

```
[6]: print("Dataset loaded successfully!")
     print("Dataset shape:", df.shape)
     print("\nFirst five rows:")
     print(df.head())
```

```
Dataset loaded successfully!
Dataset shape: (569, 31)

First five rows:
  diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0         M        17.99         10.38          122.80     1001.0
1         M        20.57         17.77          132.90     1326.0
2         M        19.69         21.25          130.00     1203.0
3         M        11.42         20.38           77.58      386.1
4         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   symmetry_mean  ...  radius_worst  texture_worst  perimeter_worst  \
0         0.2419  ...         25.38          17.33           184.60
1         0.1812  ...         24.99          23.41           158.80
2         0.2069  ...         23.57          25.53           152.50
3         0.2597  ...         14.91          26.50            98.87
4         0.1809  ...         22.54          16.67           152.20

   area_worst  smoothness_worst  compactness_worst  concavity_worst  \
0      2019.0            0.1622             0.6656           0.7119
1      1956.0            0.1238             0.1866           0.2416
2      1709.0            0.1444             0.4245           0.4504
3       567.7            0.2098             0.8663           0.6869
4      1575.0            0.1374             0.2050           0.4000

   concave points_worst  symmetry_worst  fractal_dimension_worst
0                0.2654          0.4601                  0.11890
1                0.1860          0.2750                  0.08902
2                0.2430          0.3613                  0.08758
3                0.2575          0.6638                  0.17300
4                0.1625          0.2364                  0.07678

[5 rows x 31 columns]
```

```
[7]: y = df["diagnosis"].map({"M": 1, "B": 0}).to_numpy()
     X = df.drop(columns=["diagnosis"]).to_numpy(dtype=float)

[8]: np.random.seed(42)

     indices = np.random.permutation(len(X))
     split_index = int(0.80 * len(X))

     train_indices = indices[:split_index]
     test_indices = indices[split_index:]

[9]: X_train = X[train_indices]
     X_test = X[test_indices]
     y_train = y[train_indices]
     y_test = y[test_indices]

[10]: mean = X_train.mean(axis=0)
      std = X_train.std(axis=0)
```

[13]:
```
std[std == 0] = 1

X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

print("\nPreprocessing completed.")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Number of features:", X_train.shape[1])
```

```
Preprocessing completed.
Training samples: 455
Testing samples: 114
Number of features: 30
```

[14]:
```
class SingleLayerPerceptron:

    def __init__(self, learning_rate=0.01, epochs=100):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = 0.0
        self.errors = []

    # Step activation function
    def activation(self, value):
        return 1 if value >= 0 else 0

    # Train the perceptron
    def fit(self, X, y):

        number_of_samples, number_of_features = X.shape

        self.weights = np.zeros(number_of_features)
        self.bias = 0.0

        for epoch in range(self.epochs):
            total_errors = 0

            # Shuffle training samples each epoch
            order = np.random.permutation(number_of_samples)

            for i in order:
                # Weighted sum plus bias
                z = np.dot(X[i], self.weights) + self.bias

                # Predicted class
                prediction = self.activation(z)
                # Calculate error
                error = y[i] - prediction

                # Update weights and bias
                self.weights += (
                    self.learning_rate * error * X[i]
                )
                self.bias += self.learning_rate * error

                if error != 0:
                    total_errors += 1

            self.errors.append(total_errors)

            if (epoch + 1) % 10 == 0:
                print(
                    f"Epoch {epoch + 1}/{self.epochs}, "
                    f"Errors: {total_errors}"
                )

    # Predict class labels
    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return (linear_output >= 0).astype(int)
```

[15]:
```
model = SingleLayerPerceptron(
    learning_rate=0.01,
    epochs=100
)

print("\nTraining started...")
model.fit(X_train, y_train)
print("Training completed!")
```

```
Training started...
Epoch 10/100, Errors: 49
Epoch 20/100, Errors: 47
Epoch 30/100, Errors: 41
Epoch 40/100, Errors: 52
Epoch 50/100, Errors: 33
Epoch 60/100, Errors: 36
Epoch 70/100, Errors: 32
Epoch 80/100, Errors: 39
Epoch 90/100, Errors: 38
Epoch 100/100, Errors: 31
Training completed!
```

[16]:
```
y_pred = model.predict(X_test)
```

[17]:
```
TP = np.sum((y_test == 1) & (y_pred == 1))
TN = np.sum((y_test == 0) & (y_pred == 0))
FP = np.sum((y_test == 0) & (y_pred == 1))
FN = np.sum((y_test == 1) & (y_pred == 0))

accuracy = (TP + TN) / len(y_test)

precision = TP / (TP + FP) if (TP + FP) else 0
recall = TP / (TP + FN) if (TP + FN) else 0

f1_score = (
    2 * precision * recall / (precision + recall)
    if (precision + recall) else 0
)

print("\n========== FINAL RESULTS ==========")
print("True Positives (TP):", TP)
print("True Negatives (TN):", TN)
print("False Positives (FP):", FP)
print("False Negatives (FN):", FN)

print(f"\nAccuracy:  {accuracy * 100:.2f}%")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-Score:  {f1_score:.4f}")

print("\nConfusion Matrix:")
print(np.array([[TN, FP],
                [FN, TP]]))
```

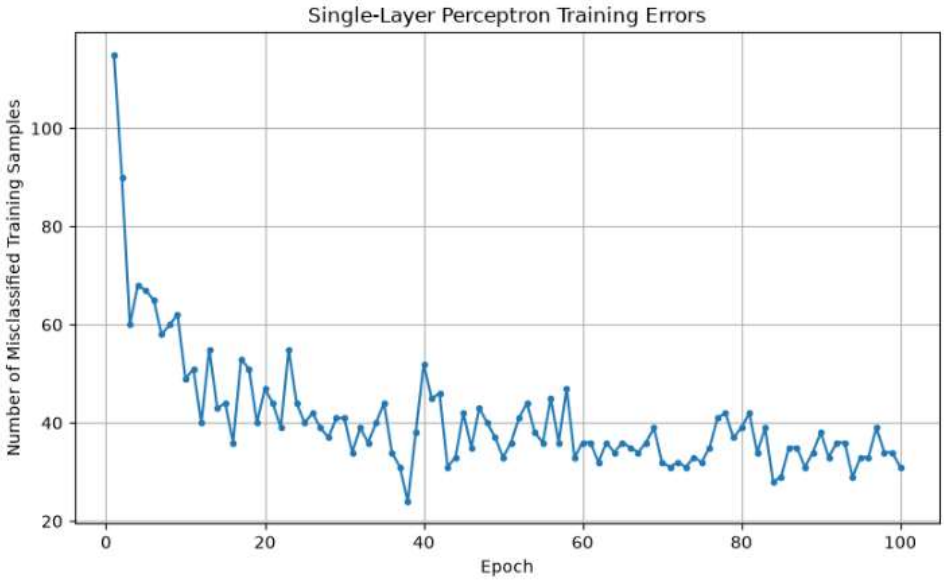
```
========== FINAL RESULTS ==========
True Positives (TP): 44
True Negatives (TN): 64
False Positives (FP): 3
False Negatives (FN): 3

Accuracy:  94.74%
Precision: 0.9362
Recall:    0.9362
F1-Score:  0.9362

Confusion Matrix:
[[64  3]
 [ 3 44]]
```

[18]:
```
plt.figure(figsize=(8, 5))
plt.plot(
    range(1, model.epochs + 1),
    model.errors,
    marker='o',
    markersize=3
)

plt.title("Single-Layer Perceptron Training Errors")
plt.xlabel("Epoch")
plt.ylabel("Number of Misclassified Training Samples")
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
[18]: print("\nSample Predictions:")
      print("Actual | Predicted")

      for actual, predicted in zip(y_test[:10], y_pred[:10]):
          actual_label = "Malignant" if actual == 1 else "Benign"
          predicted_label = "Malignant" if predicted == 1 else "Benign"

          print(f"{actual_label:10} | {predicted_label}")
```

```
Sample Predictions:
Actual | Predicted
Malignant  | Malignant
Malignant  | Malignant
Benign     | Benign
Malignant  | Malignant
Malignant  | Malignant
Benign     | Benign
Benign     | Benign
Benign     | Benign
Malignant  | Malignant
Benign     | Benign
```